Prometheus

COMPLETE NOTES

From Basics to Advanced



Metrics Collection

- Pull Model
- Exporters
- Targets



Visualization with Grafana

- Dashboards
- Graphs
- Alert Panels
- Real-time Data

NEERAJ SINGH
DEVOPS



PromQL Queries

- Instant Query
- Range Query
- Functions
- Real-time Analysis

"PROMETHEUS" for PDF





Alerting & Monitoring

- Alert Rules
- AlertManager
- Notifications
- Real-time Alerts



Real-world Examples

- Kubernetes
- Docker
- Linux Servers
- Microservices



Best Practices

- Monitoring Strategy
- Scaling
- Security
- Performance Optimization



Interview Questions

- Concepts
- Scenario-based
- Practical Questions
- Answers with Explanation



Production Use Cases

- Application Monitoring
- Infrastructure Monitoring
- Kubernetes Monitoring
- Alerting & Troubleshooting

Better Monitoring → More Reliable Systems ❤

Before Prometheus ... Understand Observability

What is Observability?

- Ability to understand what is happening inside your systems using data.
- Helps you find, fix and prevent issues in production.
- Gives visibility into system behavior (applications, infrastructure, services).

The 3 Pillars

Metrics
- What happened?
- Numerical data
- Time-series
- E.g. CPU, Memory, Request count

Logs
- Why happened?
- Detailed information
- Text data
- E.g. Application logs, error logs

Traces
- Where happened?
- Request flow across services
- Shows the complete journey of a request

4 Golden Signals

Latency
- How fast is your system responding?
- e.g. response time (ms)

Traffic
- How much traffic is your system handling?
- e.g. requests per second (RPS)

Errors
- How many requests are failing?
- e.g. error rate (4xx, 5xx)

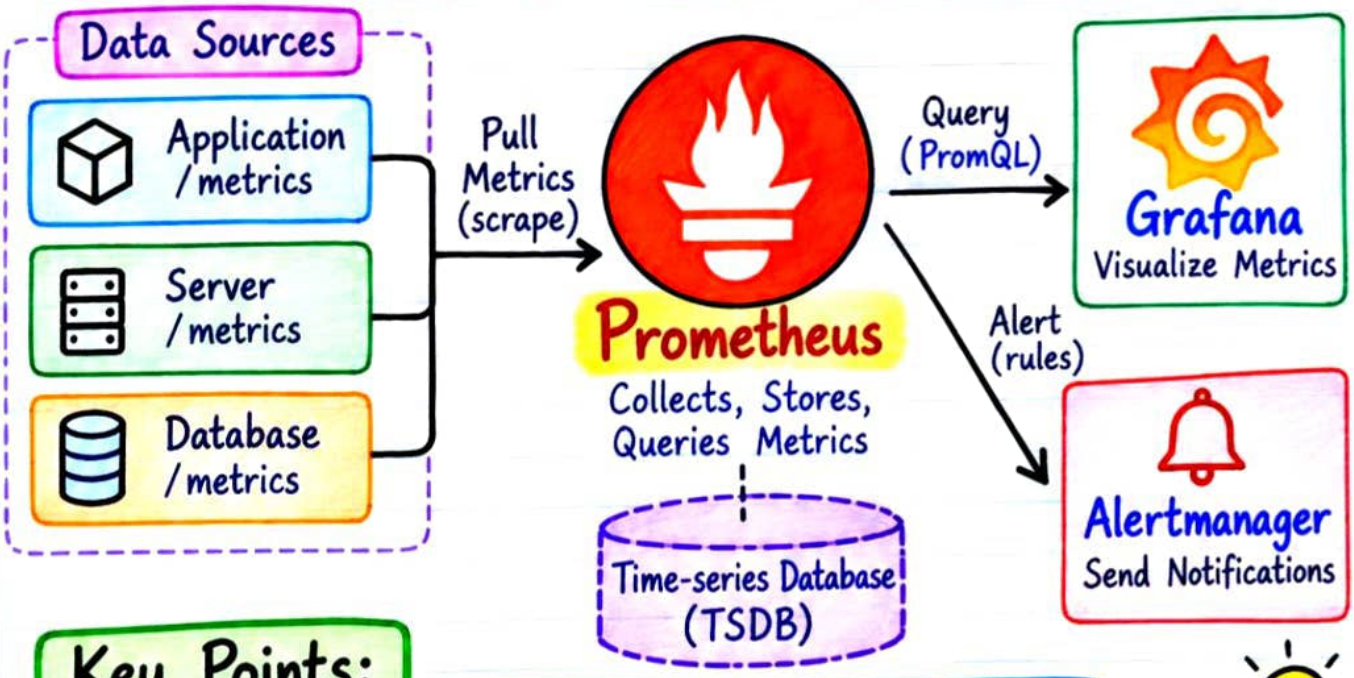
Saturation
- How close are you to your resource limits?
- e.g. CPU, Memory, Disk, Network utilization

"You can't fix what you can't see."

— Observability Matters

What is Prometheus?

Prometheus is an **open-source** monitoring and alerting system designed to collect, store and query **time-series metrics** from your infrastructure and applications.



Key Points:

- **Open-source** (CNCF project)
- Pull-based model (scrapes metrics)
- Stores data as **time-series**
- Powerful query language (**PromQL**)
- Built-in **alerting** (with Alertmanager)
- Widely used in **Kubernetes** and cloud environments

Use Cases:

- Infrastructure monitoring
- Application monitoring
- Kubernetes monitoring
- Set alerts for critical issues
- Track SLIs and SLOs
- Observability for DevOps teams

Prometheus = Collect + Store + Query + Alert

Metrics Power Better Decisions ♥

Prometheus Architecture

How all the components work together ?

Targets
(Applications / Infrastructure)

- Servers (Node Exporter)
- Containers (cAdvisor)
- Kubernetes (kube-state-metrics)
- Databases (MySQL, Postgres)
- Applications (/metrics endpoint)

Scrape metrics (pull)

Service Discovery
(Kubernetes, DNS, etc.)

- Find targets automatically
- Add / remove targets

Find Targets

Prometheus Server

- Collects metrics (pull model)
- Stores data (TSDB)
- Queries using PromQL

Send Alerts

Alertmanager

- Grouping
- Routing
- Inhibition
- Silences

Notifications

- Email
- Slack
- Webhook

(and more...)

Query (PromQL)

Grafana

- Query metrics (PromQL)
- Visualize data
- Create dashboards
- Set alerts (via Alerting)

Time-series Database (TSDB)

- Stores metrics
- Indexed by labels
- Retention policy

Key Takeaways:

- Prometheus uses a pull-based model (scrapes metrics)
- Discovers targets via service discovery
- Stores data in a time-series database (TSDB)
- Supports powerful querying with PromQL
- Uses Alertmanager for notifications
- Widely used in Kubernetes and cloud environments

Data Flow:

1. Discover targets (service discovery)
2. Scrape metrics from targets
3. Store metrics in TSDB
4. Query metrics using PromQL
5. Send alerts to Alertmanager
6. Visualize metrics in Grafana

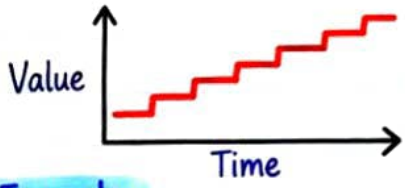
Monitor Today for a Better Tomorrow

Prometheus Metric Types

Prometheus supports **4 types** of metrics to capture different kinds of data from your systems and applications.

1 Counter

- Only increases (never decreases)
- Used to count things
- Resets to 0 on restart



Used for counting events like requests, errors, etc.

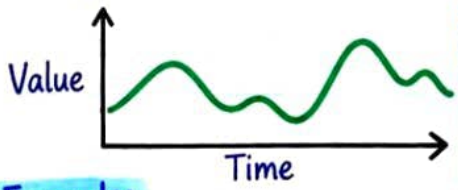
Example:

`http_requests_total`

Total number of HTTP requests

2 Gauge

- Can increase or decrease
- Used for values that change over time
- Represents a current state



Used for values that can go up and down like memory, CPU, etc.

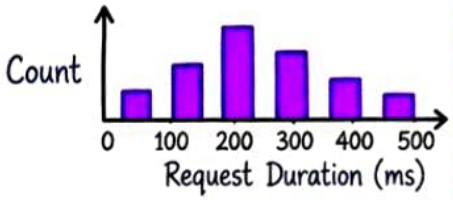
Example:

`node_memory_usage_bytes`

Current memory usage

3 Histogram

- Observations are grouped into buckets
- Useful for understanding the distribution of values
- Provides count, sum and bucket metrics



Used to measure latency or request duration.

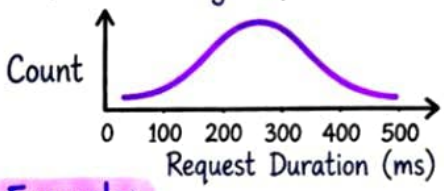
Example:

`http_request_duration_seconds`

Tracks request duration in buckets

4 Summary

- Also tracks the distribution of values
- Calculates quantiles (e.g., 50th, 90th, 99th)
- Computed on the client side
- Does not expose buckets (unlike histogram)



Used to get quantile values like 0.5, 0.9, 0.99 for latency.

Example:

`http_request_duration_seconds`

Provides quantiles like 0.5, 0.9, 0.99

Key Takeaways:

- Use **Counter** for counting events.
- Use **Gauge** for values that go up and down.
- Use **Histogram/Summary** for measuring latency or size distribution.
- Choose the right metric type for accurate monitoring.

Interview Tip:

What is the difference between Histogram and Summary?

- Histogram exposes buckets (count, sum, buckets).
- Summary provides quantiles directly.
- Histogram is better for aggregation across instances, while Summary is client-side.

Metrics Tell the Story

Exporters & Targets in Prometheus

Prometheus collects metrics from different systems using **exporters** or by scraping **/metrics** endpoints.

What is an Exporter?

- An exporter exposes system metrics in a format Prometheus can scrape (**/metrics**).
- It collects data from a service and makes it available over HTTP.
- Each exporter runs as a separate process.

Node Exporter
- Linux Metrics

App /metrics
- Application Metrics

Database Exporter
- MySQL/ Postgres Metrics

Blackbox Exporter
- Endpoint Monitoring

Prometheus
Scrapes Metrics from Targets

Grafana
- Visualize Metrics
- (Create Dashboards)

DevOps / SRE Teams

Common Exporters

1. Node Exporter – Linux server metrics
2. Blackbox Exporter – HTTP/HTTPS/TCP checks
3. Windows Exporter – Windows metrics
4. MySQL Exporter – MySQL database metrics
5. PostgreSQL Exporter – PostgreSQL metrics
6. cAdvisor – Container metrics
7. kube-state-metrics – Kubernetes object metrics
8. Custom Exporter – Application-specific metrics

What is a Target?

- A target is an endpoint that Prometheus scrapes.
- Defined in `prometheus.yml` under a job.
- Each target has a set of labels (e.g., instance, job).
- A **job** is a logical group of similar targets.

Key Points:

- Exporters expose **/metrics** endpoint.
- Prometheus scrapes targets (pull model).
- Each target belongs to a job.
- Use exporters for infrastructure, databases, applications and external endpoints.
- You can also expose metrics directly from your application.

More Metrics = Better Visibility = More Reliable Systems ♥

prometheus.yml

The Heart of Prometheus Configuration

The prometheus.yml file tells Prometheus what to scrape, how often to scrape and which targets to monitor.

Example: prometheus.yml

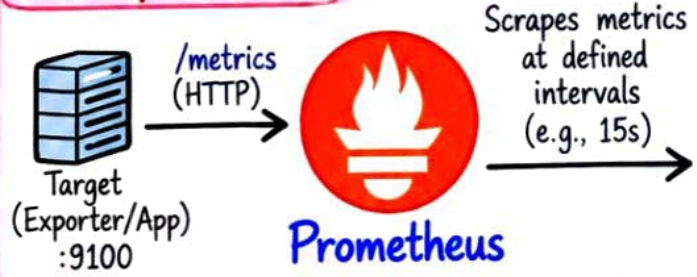
```yaml
global:
  scrape_interval: 15s  # how often to scrape
  evaluation_interval: 15s  # how often to evaluate
scrape_configs:
  - job_name: "node"  # job name
    static_configs:  # define targets
      - targets:
          - "localhost:9100"  # node exporter

  - job_name: "app"
    static_configs:
      - targets:
          - "app-server:8080" # application metrics
```

Key Configuration Options

1. **global** – global settings for Prometheus
   - scrape_interval – how often to collect metrics
   - evaluation_interval – how often to evaluate rules
2. **scrape_configs** – list of jobs to scrape
3. **job_name** – logical name for the target group
4. **static_configs** – manually define targets
5. **targets** – endpoints (IP:PORT) to scrape metrics
6. **metrics_path** – default /metrics (can be changed)
7. **scheme** – http or https
8. **relabel_configs** – modify or filter labels
9. **service discovery** – dynamic target discovery (Kubernetes, EC2, Consul, DNS, etc.)

Scrape Flow



Service Discovery (Dynamic Targets)

- Kubernetes – uses Kubernetes API
- EC2 – discovers AWS instances
- Consul – service discovery
- DNS – via DNS records
- File-based – from a targets file

Automatically finds and monitors targets without manual updates.

Tips & Best Practices

- Use meaningful job names (e.g., node, app, db)
- Set appropriate scrape_interval (e.g., 15s, 30s)
- Use service discovery in production
- Use relabeling to add useful labels (env, service, etc.)
- Monitor Prometheus itself (self-scraping)
- Keep scrape targets healthy and reachable

Common Mistakes

- Wrong target port
- Exporter not running
- Using too short scrape_interval (high load)
- Missing or incorrect /metrics endpoint
- Not using labels effectively
- Scraping too many unnecessary targets

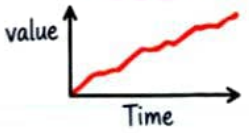
Good Configuration = Reliable Monitoring = Peace of Mind

PromQL Cheat Sheet

PromQL is the **query language** of Prometheus to query, filter, aggregate and analyze **time-series metrics**.

1 rate()

- Returns per-second rate of increase
- Used with counters
- Useful for calculating request rate, error rate etc.



Example:

```
rate(http_requests_total[5m])
  # Requests per second
```

2 increase()

- Total increase in a time range
- Used with counters
- Useful to find total number of events in a given time

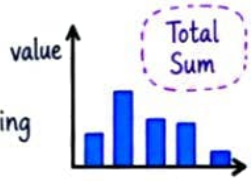


Example:

```
increase(http_requests_total[1h])
  # Total requests in 1 hour
```

3 sum()

- Adds up values
- Often used with by() for grouping
- Useful to get total metrics across instances

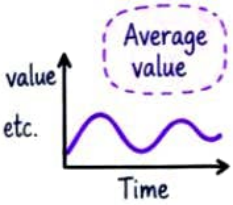


Example:

```
sum(rate(http_requests_total[5m]))
    by (service)
  # Requests per service
```

4 avg()

- Calculates average value
- Used for latency, response time, etc.

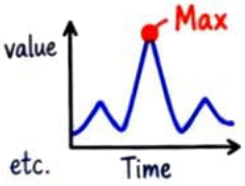


Example:

```
avg(http_request_duration_seconds)
  # Average request duration
```

5 max()

- Returns the maximum value
- Useful for finding spikes
- Used for CPU, memory, latency, etc.

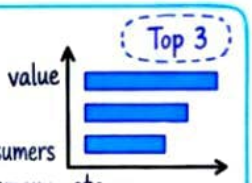


Example:

```
max(node_cpu_usage_percent)
  # Max CPU usage
```

6 topk()

- Returns top N time series
- Useful to find top resource consumers
- Commonly used for top CPU, memory, etc.

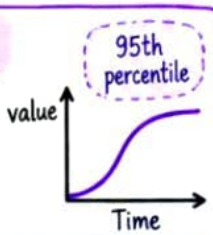


Example:

```
topk(3, sum(rate(container_cpu_usage_seconds_total[5m])) by (pod))
  # Top 3 pods by CPU usage
```

7 histogram_quantile()

- Calculates percentile from histogram
- Used for latency (e.g., 90th, 95th)
- Useful for SLO/SLA monitoring



Example:

```
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket[5m]))
    by (le))
  # 95th percentile latency
```

Common Filters

Filter by label	`metric{job="node"}`
Multiple labels	`metric{job="node", env="prod"}`
Regex match	`metric{instance=~".*:9100"}`
Exclude	`metric{job!="batch"}`

Key Takeaways

- Use the right function for the right use case
- Combine functions with label filters (by, without)
- Practice with real metrics from your environment
- PromQL is powerful for troubleshooting and dashboards

Good Queries Lead to Better Insights ♥

Recording & Alerting Rules

Turn Your Metrics Into Actions

- Monitor
- Detect Issues
- Take Action
- Keep Systems Healthy

1 Recording Rules

- Pre-compute and store query results as a new time series.
- Makes complex queries faster.
- Useful for dashboards and alerting.
- Reduces query load on Prometheus.

Example: recording_rules.yml

```
groups:
  - name: cpu-recording-rules
    rules:
      - record: node:cpu_usage:rate5m
        expr: rate(node_cpu_seconds_total[5m])
```

Creates a new metric which can be reused in dashboards and alerts.

2 Alerting Rules

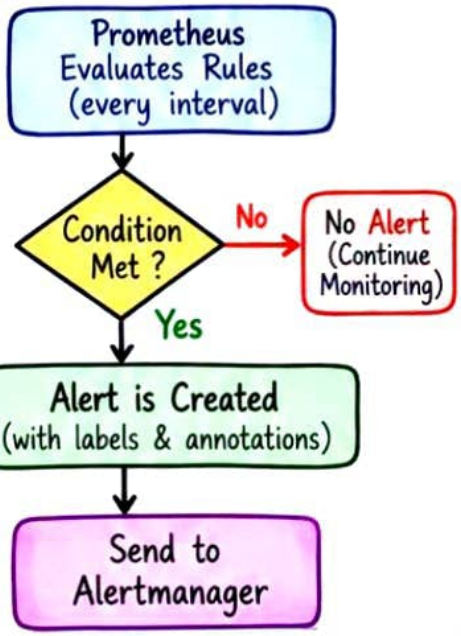
- Evaluate PromQL expressions at regular intervals.
- Trigger an alert when a condition is met.
- Send alerts to Alertmanager.
- Helps detect issues before users are impacted.

Example: alert_rules.yml

```
groups:
  - name: cpu-alerts
    rules:
      - alert: HighCPUUsage
        expr: rate(node_cpu_seconds_total[5m]) > 80
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High CPU usage detected"
          description: "CPU usage is above 80% for 5 minutes"
```

Triggers an alert when CPU usage is above 80% for 5 minutes.

3 Rule Evaluation Flow



4 Alerting Rule States

- **Inactive** – condition is false.
- **Pending** – condition is true, but waiting for 'for' duration.
- **Firing** – condition is true and alert is active.

5 Best Practices

- Use meaningful alert names
- Set appropriate thresholds
- Use 'for' to avoid false alerts
- Add clear labels and annotations
- Alert on symptoms, not noise
- Test your alerting rules
- Use recording rules for heavy queries

6 Common Mistakes

- Alerting on short spikes
- No 'for' duration
- Missing or unclear annotations
- Too many low-priority alerts
- Wrong thresholds
- Not testing alerts

7 Interview Tip

What is the difference between Recording Rules and Alerting Rules?

- **Recording Rules** = optimization (pre-compute and reuse metrics)
- **Alerting Rules** = action (trigger alerts based on conditions)

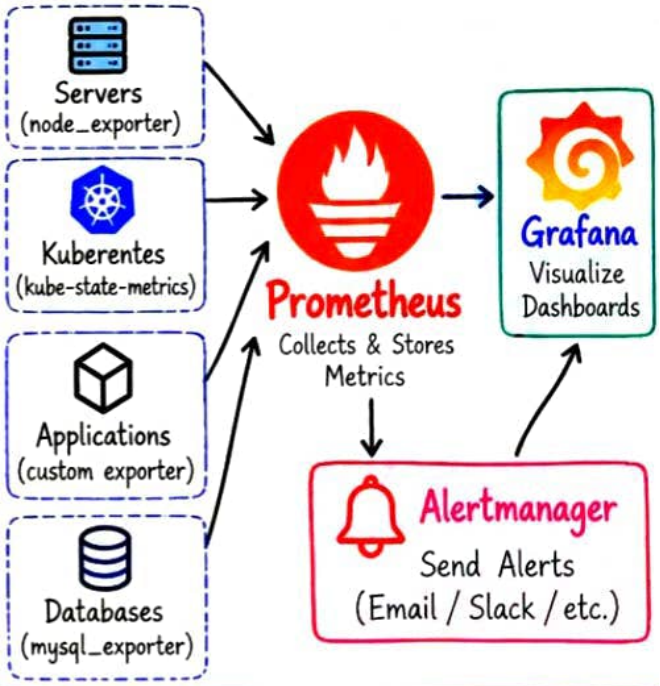
Good Monitoring Rules = Faster Insights = More Reliable Systems ♥

Prometheus
Key Takeaways & Next Steps

1 Key Takeaways

- Prometheus collects metrics using **pull** model.
- Supports 4 metric types: **Counter, Gauge, Histogram, Summary.**
- Uses **exporters** to collect metrics from various systems.
- Targets are defined in **prometheus.yml.**
- Powerful query language (**PromQL**) to analyze data.
- Works great with **Grafana** for visualization.
- **Alerting rules** help detect and fix issues early.
- Lightweight, reliable and widely used in **DevOps** and **SRE** teams.

2 Prometheus in DevOps Ecosystem



3 Useful Links

- Prometheus: https://prometheus.io
- Documentation: https://prometheus.io/docs/
- PromQL Cheatsheet: https://promlabs.com/
- Exporters: https://prometheus.io/docs/instrumenting/
- Grafana: https://grafana.com
- Alertmanager: https://prometheus.io/docs/alerting/

4 Next Steps

1. Set up Prometheus in a local environment.
2. Configure node_exporter and explore metrics.
3. Write your own PromQL queries.
4. Build dashboards in Grafana.
5. Create alerting rules for real-world scenarios.
6. Try monitoring Kubernetes, applications and databases.
7. Explore advanced topics like federation, remote storage and high availability.

5 Interview Questions

- What is Prometheus and how does it work?
- Explain the 4 types of Prometheus metrics.
- What is the difference between Counter and Gauge?
- How do you configure targets in Prometheus?
- How does Prometheus handle high availability?

6 Final Note

Metrics tell the story of your system.
Prometheus helps you listen, understand and take action.

Monitor → Analyze → Alert → Improve

Better Observability. Better Systems. Happier Teams.

Keep Learning Keep Exploring Keep Building ♥

Prometheus

Complete Series Wrap Up

From Metrics to Meaningful Insights

1 What We Learned?

1. What is Prometheus?
2. How it works (architecture)
3. prometheus.yml configuration
4. Exporters and targets
5. PromQL basics and useful queries
6. Recording and Alerting rules
7. Visualization with Grafana
8. Best practices and common mistakes
9. Real-world examples
10. Next steps and career tips

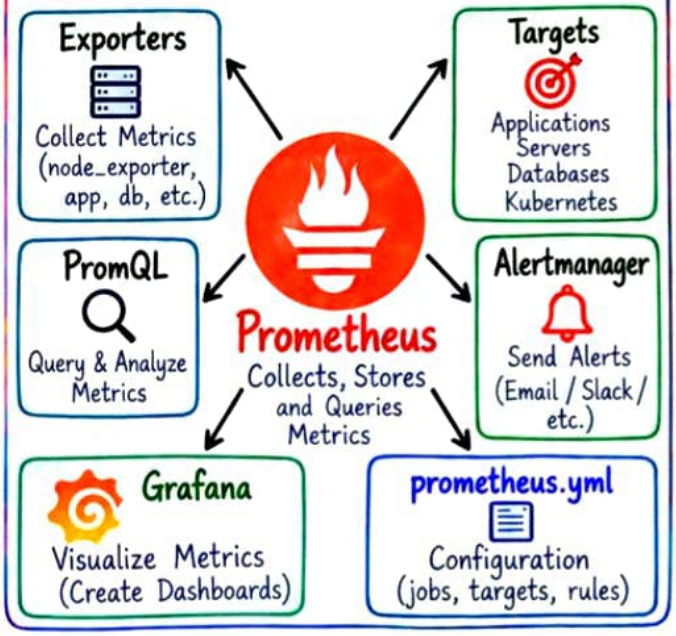
2 Why Prometheus is Important?

- Open source and CNCF project
- Lightweight and scalable
- Powerful query language (PromQL)
- Works with multiple systems (K8s, AWS, etc.)
- Helps in proactive monitoring and alerting
- Widely used in DevOps and SRE roles
- Great community support

3 Real-World Impact

- Detect issues before users are impacted
- Improve system reliability and uptime
- Make data-driven decisions
- Reduce downtime and MTTR
- Essential for modern cloud-native environments

4 Prometheus Ecosystem Recap



5 Your Next Steps

1. Set up Prometheus in your environment
2. Monitor real applications (e.g., Kubernetes)
3. Create custom dashboards in Grafana
4. Configure meaningful alerting rules
5. Explore advanced topics (federation, HA, remote storage)
6. Work on real-world projects
7. Keep learning and stay updated

Monitor (Collect Metrics) → **Understand** (Analyze Data) → **Take Action** (Alerts & Fix Issues) → **Grow** (Reliable Systems)

Observability Today → Reliable Systems Tomorrow

Prometheus Final Thoughts

Monitor Today **Secure Tomorrow**

1 Key Benefits Recap

- Open source and CNCF project
- Lightweight and scalable
- Powerful PromQL for flexible queries
- Works well in cloud-native environments
- Multiple exporters and target support
- Great integration with Grafana
- Effective alerting and recording rules
- Widely adopted in DevOps and SRE teams
- Active community and continuous improvement

2 Common Use Cases

1. Monitor Kubernetes clusters
2. Track application performance
3. Monitor infrastructure (servers, VMs, containers)
4. Set up alerts for critical metrics
5. Visualize metrics in Grafana dashboards
6. Detect anomalies and troubleshoot issues
7. Ensure high availability and reliability

3 Pro Tips for Success

- Start with basic setup and simple queries
- Monitor your own infrastructure first
- Use meaningful labels and naming conventions
- Build dashboards step by step
- Set up useful alerts (avoid alert fatigue)
- Learn from real-world scenarios
- Keep exploring and contribute to the community

4 Who Should Learn Prometheus ?

- DevOps Engineers
- SRE (Site Reliability Engineers)
- Cloud Engineers
- Kubernetes Administrators
- System Administrators
- Developers (for observability)
- Anyone interested in monitoring and reliability

Great Skill for Your Career

5 Metrics Tell a Story

Metrics don't just show numbers... They tell the story of your system



6 Keep Learning

- Explore advanced PromQL functions
- Try Thanos / Cortex for long-term storage
- Learn service discovery in depth
- Integrate with more tools (Alertmanager, Grafana, K8s, etc.)
- Work on real-time projects
- Stay updated with new features and releases

Observe → Understand → Improve → Build Better Systems

Thank You for Reading ! Keep Learning Keep Growing

Prometheus

Bonus Resources

Learn | Practice | Explore | Grow

1 Official Links

- Prometheus: https://prometheus.io
- Documentation: https://prometheus.io/docs/
- PromQL Guide: https://prometheus.io/docs/prometheus/latest/querying/basics/
- Configuration: https://prometheus.io/docs/prometheus/latest/configuration/

Start Here [Learn Basics]

2 Hands-on Practice

- PromLabs: https://promlabs.com (Hands-on labs for Prometheus)
- Play with Prometheus: https://play-with-prometheus.com (Practice in browser)
- Katacoda (scenarios): https://www.katacoda.com (Interactive scenarios)
- Kubernetes with Prometheus (kube-prometheus): https://github.com/prometheus-operator/kube-prometheus

Practice Build Get Confident

3 Grafana Dashboards

- Grafana: https://grafana.com
- Prometheus Dashboard Templates: https://grafana.com/grafana/dashboards/ (Use ready-made dashboards)
- Node Exporter Dashboard (ID: 1860)
- Kubernetes Cluster Dashboard (ID: 315)
- Application Dashboard (Spring Boot, Nginx, etc.)

Ready Dashboards for Quick Start

4 Sample Projects

- Monitor a Linux server (node_exporter)
- Monitor a Kubernetes cluster (kube-state-metrics)
- Monitor a Django / Spring Boot app
- Create custom alerts
- Build a complete monitoring stack with Prometheus + Grafana + Alertmanager
- Try real-world scenarios (EKS, GKE, EC2, etc.)

Learn by Doing

5 Community & Support

- Prometheus GitHub: https://github.com/prometheus/prometheus/
- CNCF: https://www.cncf.io/projects/prometheus/
- Prometheus Community: https://discuss.prometheus.io
- Slack: https://cloud-native.slack.com (#prometheus)
- Reddit: r/prometheus

Be a Part of the Community

6 Recommended Learning Path

1. Learn the basics (architecture, metrics, scraping)
2. Practice PromQL with real data
3. Set up alerts and notifications
4. Integrate with Grafana
5. Work on real-world projects
6. Explore advanced topics (federation, HA, remote storage)
7. Contribute to open source

Step by Step to Mastery

7 Final Message

"Monitoring is not just about metrics, it's about the health, reliability and success of your systems."

Keep Learning
Keep Building
You Got This!

Better Monitoring → Better Systems → Brighter Future

Prometheus Interview Questions

Be Ready Be Confident You Can Do It !

PREPARE
PRACTICE
SUCCEED

1 What is Prometheus ?

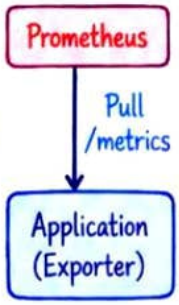
- Open source monitoring and alerting tool.
- Collects metrics using pull model.
- Stores time series data.
- Widely used in Kubernetes and cloud-native environments.

Prometheus

Used for monitoring applications, infrastructure and services in real-time.

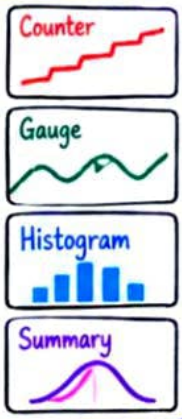
2 Why Pull Model ?

- Prometheus scrapes metrics from endpoints at regular intervals.
- More reliable and avoids pushing complexity to applications.
- Targets expose metrics via /metrics endpoint.
- Simple and scalable architecture.



3 What are the 4 types of metrics ?

1. **Counter** – monotonically increasing value (e.g., http_requests_total)
2. **Gauge** – value can go up or down (e.g., node_memory_usage_bytes)
3. **Histogram** – samples observations (e.g., request_duration_seconds)
4. **Summary** – similar to histogram but calculates quantiles client-side



4 How does PromQL work ?

- PromQL is the query language of Prometheus.
- Used to query and analyze time series data.
- Supports filtering, aggregation, functions and operators.
- Helps to extract meaningful insights.

Example:

```
rate(http_requests_total[5m])
# Calculate per second request rate over 5 minutes
```

- ✓ Query
- ✓ Analyze
- ✓ Get Insights

5 How do you configure alerts ?

- Write alerting rules in YAML file.
- Define condition, duration (for) and labels.
- Alertmanager sends notifications (email, Slack, etc.).
- Helps to detect and fix issues early.

Example:

```
alert: HighCPUUsage
expr: rate(node_cpu_seconds_total[5m]) > 80
for: 5m
labels:
  severity: warning
annotations:
  summary: "High CPU usage detected"
  description: "CPU usage is above 80% for 5 minutes"
```

Detect Issues Early

6 How do you visualize metrics ?

- Use Grafana to create dashboards.
- Add Prometheus as a data source.
- Use PromQL queries to build panels.
- Visualize metrics like CPU, memory, request rate, latency, etc.
- Create meaningful dashboards for monitoring and troubleshooting.

Grafana

Grafana Dashboards = Better Insights

7 Any tips for the interview ?

- ✓ Understand core concepts (scraping, storage, PromQL)
- ✓ Practice real queries
- ✓ Work on hands-on projects
- ✓ Learn alerting and Grafana dashboards
- ✓ Be ready for real-world scenarios
- ✓ Explain with examples and use cases

Keep Learning
Keep Growing
All the Best !

Monitoring Today → Reliable Systems → Brighter Future

Prometheus Complete Series

What's Next ?



Take Your Monitoring Skills to the Next Level !

1 Tools to Learn Next

- **Grafana** – Visualization (already started)
- **Alertmanager** – Alerting and notifications
- **Loki** – Log aggregation (logs)
- **Tempo** – Distributed tracing (traces)
- **Jaeger** – Tracing alternative
- **cAdvisor** – Container metrics
- **Node Exporter** – System metrics
- **Kube State Metrics** – Kubernetes metrics
- **Thanos / Cortex** – Long-term storage
- **OpenTelemetry** – Metrics, logs and traces (observability)

2 Real-World Projects

- ✅ Set up complete monitoring stack (Prometheus + Grafana + Alertmanager)
- ✅ Monitor Kubernetes cluster (EKS / AKS / GKE)
- ✅ Monitor a real application (Spring Boot / Node.js)
- ✅ Create custom metrics and dashboards
- ✅ Set up alerting for critical scenarios (CPU, memory, pod down, HTTP errors)
- ✅ Implement long-term storage with Thanos
- ✅ Build an end-to-end observability stack (Prometheus + Grafana + Loki + Tempo)

3 Career Opportunities

- DevOps Engineer
- SRE (Site Reliability Engineer)
- Cloud Engineer
- Kubernetes Administrator
- Observability / Monitoring Specialist
- Platform Engineer
- Solutions Architect



Better Skills
Bigger
Opportunities

4 Keep Learning



Learning
Never
Stops

- Practice with real environments
- Explore CNCF projects
- Contribute to open source
- Follow community blogs and meetups
- Work on hands-on labs (Kubernetes, Play with K8s)
- Stay updated with new features and releases
- Build your own monitoring setups

5 Final Words



"Monitoring is not a destination, it's a continuous journey to build reliable and scalable systems."



Observe → Understand → Improve → Grow



You're on the Right Path !

Same Curiosity Same Learning Same You → Bigger Future ❤